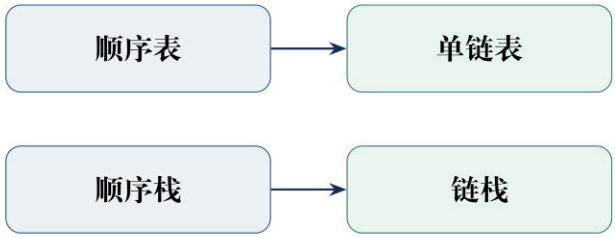
DATA STRUCTURES & ALGORITHMS

题目精讲 与超详细解析

C++ 实现 · 算法推导 · 复杂度分析 · 测试验证



整理日期：2026 年 9 月 15 日

目录

第一章　全部原始题目 **2**

1.1　实验 1.1：线性表的操作 2

1.2　实验 1.2：栈的操作 2

第二章　全部答案与详细解析 **4**

2.1　实验 1.1：线性表的操作 4

2.1.1　考查知识点与总体设计 4

2.1.2　数据结构示意图 4

2.1.3　顺序表基本操作：逐项推导 5

2.1.4　链表基本操作：逐项推导 6

2.1.5　复杂度对比 8

2.1.6　易错点 8

2.1.7　结果检验与测试设计 9

2.1.8　实验 1.1 完整 C++ 源码 9

2.2　实验 1.2：栈的操作 16

2.2.1　考查知识点与总体设计 16

2.2.2　顺序栈与链栈结构 16

2.2.3　基本操作逐项说明 16

2.2.4　扩展 1：十进制整数转八进制 17

2.2.5　括号匹配：为什么一定适合用栈？ 17

2.2.6　中缀表达式与后缀表达式 18

2.2.7　中缀转后缀：运算符栈算法 18

2.2.8　后缀表达式求值 19

2.2.9　异常输入与健壮性处理 20

2.2.10　一题多解：表达式计算的两种栈方案 20

2.2.11　复杂度汇总 21

2.2.12　结果检验 21

2.2.13　实验 1.2 完整 C++ 源码 21

第三章　参考资料与核验说明 **29**

第 1 章

全部原始题目

1.1 实验 1.1：线性表的操作

分别利用线性表的**顺序存储结构**和**链式存储结构**实现教材信息的管理。教材信息包含 ISBN、出版社、作者、价格等信息。

（1）顺序存储结构

在线性表顺序存储结构下实现基本操作，包括：

1. 初始化；
2. 建表；
3. 取值；
4. 查找；
5. 插入；
6. 删除；
7. 两个非递减有序表的归并等。

（2）链式存储结构

在线性表链式存储结构下实现基本操作，包括：

1. 初始化；
2. 建表；
3. 取值；
4. 查找；
5. 插入；
6. 删除；
7. 两个非递减有序链表的归并等。

1.2 实验 1.2：栈的操作

实现一个简单的表达式计算器，包含以下功能。

（1）基本栈操作的实现

- 实现顺序栈的基本操作：初始化、入栈、出栈、取栈顶元素、判空；
- 实现链式栈的基本操作：初始化、入栈、出栈、取栈顶元素、判空。

（2）括号匹配检查

- 检查表达式中的小括号 () 是否正确匹配；
- 扩展：检查多种括号 ()、[]、{} 的匹配情况。

（3）中缀表达式转换与计算

- 实现中缀表达式转换为后缀表达式；
- 实现后缀表达式的计算；
- 支持四则运算 +、-、*、/。

第 2 章

全部答案与详细解析

2.1 实验 1.1：线性表的操作

2.1.1 考查知识点与总体设计

本实验本质上考查同一个抽象数据类型（List ADT）在两种物理存储方式下的实现差异：**顺序表用连续存储区保存元素；链表用结点和指针建立逻辑次序**。MIT 6.006 的链表材料指出，链式序列把每个元素保存在结点中，并通过 `next` 保存下一结点地址；Stanford CS106B 同样以“数据域 + next 指针”描述单链表结点。[6, 7]

实验建模约定

原题只说“非递减有序”而没有明确指定按哪一个教材字段排序。所给操作演示以 ISBN 作为主要可比较字段，因此本解答统一规定：**归并时按 ISBN 字典序非递减排列**。如果教师要求按价格或其他字段排序，只需把比较条件从 `isbn` 改为对应关键字，归并算法本身不变。

教材记录定义为

$$\texttt{Book} = (\texttt{ISBN}, \texttt{publisher}, \texttt{author}, \texttt{price}).$$

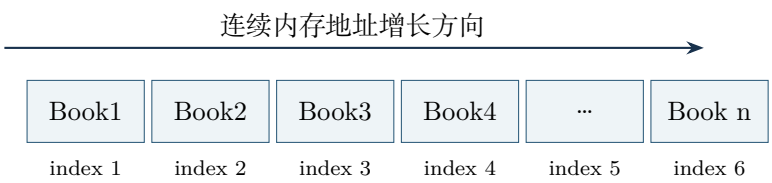
其中 ISBN 使用字符串而不是整数，原因是 ISBN 可能包含前导零或连字符；价格使用 `double`。

为什么不直接用 `std::vector` 和 `std::list`？

C++ 标准库当然已经提供 `std::vector`、`std::list` 等容器，且 `vector` 具有连续存储特性、`list` 通常实现为双向链表。[11, 12] 但本实验的目标是**亲手实现线性表的存储结构和基本操作**，所以核心结构不直接调用 STL 容器完成；源码只在非核心输出处理处少量使用标准库工具。

2.1.2 数据结构示意图

顺序表：逻辑相邻与物理相邻同时成立



顺序表可通过首地址加偏移量直接定位第 i 个元素，因此按下标取值是 $O(1)$。但在中间插入或删除时，需要整体移动后续元素，所以通常为 $O(n)$。cppreference 对连续动态数组 `std::vector` 的复杂

度说明也反映了同样的连续存储规律：随机访问为常数时间，而中间插入/删除与到尾部的距离线性相关。[11]

单链表：逻辑相邻由指针建立



链表的第 i 个元素无法通过地址公式直接得到，必须从头结点顺链访问，因此取值和按位置定位最坏为 $O(n)$。一旦已经取得插入位置的前驱结点，改变常数个指针即可完成插入/删除。MIT 6.006 对 linked-list sequence 的分析同样指出，第 i 个元素需要逐结点走访，而在已知位置附近重新链接指针是常数级操作。[6]

2.1.3 顺序表基本操作：逐项推导

1. 初始化

顺序表采用固定容量数组 `data[MAXSIZE]` 和长度变量 `length`。初始化的核心只有

$$\boxed{\texttt{length} = 0}.$$

这并不是“把整个数组清零”，因为当前有效数据范围只由 `length` 决定。未进入有效区间的旧字节不属于线性表逻辑内容。

2. 建表

建表输入 n 条教材记录并依次放入

$$\texttt{data[0]}, \texttt{data[1]}, \ldots, \texttt{data[n-1]}.$$

必须先检查 $0 \le n \le \texttt{MAXSIZE}$，否则会产生数组越界。

3. 取值

实验中采用教材常见的 **1 基位置编号**。第 i 个逻辑元素在 C++ 数组中的下标为 $i-1$：

$$\boxed{e = \texttt{data}[i-1]}, \qquad 1 \le i \le \texttt{length}.$$

因此时间复杂度是 $O(1)$。

4. 查找

按 ISBN 顺序比较：

$$\texttt{data[0].isbn}, \texttt{data[1].isbn}, \ldots$$

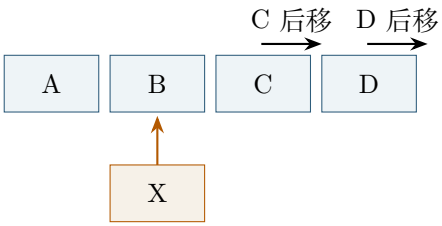
首次相等时返回位置 $i+1$，若遍历结束仍没有相等项则返回 0。普通无序表线性查找为 $O(n)$。

5. 插入

若要在第 i 个位置插入新元素 e，原来的第 $i, i+1, \ldots, n$ 个元素都要向后移动一格。移动必须**从后往前**：

$$\texttt{data[j]} \leftarrow \texttt{data[j-1]}, \qquad j = n, n-1, \ldots, i.$$

若反向从前往后移动，会用已经覆盖后的值继续覆盖后面元素，导致数据重复。



在位置 2 插入 X

最坏情况下插入表头要移动 n 个元素，因此为 $O(n)$。

6. 删除

删除第 i 个元素后，第 $i+1, \ldots, n$ 个元素依次向前移动：

$$\texttt{data[j-1]} \leftarrow \texttt{data[j]}, \qquad j = i, i+1, \ldots, n-1.$$

最后 `length--`。最坏时间复杂度同样是 $O(n)$。

7. 两个非递减顺序表归并

设 A、B 已分别按 ISBN 非递减排列。设置两个游标 i, j，每次把两边当前较小的元素复制到结果表 C：

$$\begin{cases} A_i \le B_j, & \text{取} A_i,\ i \leftarrow i+1, \\ A_i > B_j, & \text{取} B_j,\ j \leftarrow j+1. \end{cases}$$

任何一边扫描完以后，把另一边剩余元素顺序复制到 C 即可。

每个输入元素最多被比较/复制常数次，因此

$$\boxed{T(m,n) = O(m+n)}, \qquad \boxed{S(m,n) = O(m+n)}$$

（这里的空间复杂度针对本解答的独立结果表 C）。

一题多解：顺序表归并

解法一：建立独立结果表 C。优点是逻辑最直观，不破坏原表，最适合初学者和实验验收。
解法二：若结果允许覆盖某个输入表，可从尾部向前归并。当目标数组已有足够容量时，令写指针从 $m+n-1$ 开始，把较大的元素先写到尾部，可以减少额外结果数组，但下标边界更容易出错。
本实验优先采用解法一。

2.1.4 链表基本操作：逐项推导

1. 初始化与头结点

本解答采用**带头结点的单链表**：`head` 本身不保存有效教材信息，只用于统一表头插入、删除和空表情况。

初始化时执行

`head = new Node(),  head->next=nullptr,  length=0.`

带头结点的好处是删除第一个数据结点和删除中间结点可以写成同一段“前驱结点 + 目标结点”的代码。

2. 建表

本实现采用**尾插法**。维护 `tail` 指针始终指向当前最后一个结点：

`tail->next = new Node(e),  tail = tail->next.`

输入 n 条数据只需顺序走一遍，因此为 $O(n)$。若用头插法，最终顺序会与输入顺序相反，除非再做一次逆置。

3. 取值与查找

第 i 个元素必须从首元结点出发走 $i-1$ 次 `next`。因此

$$T_{\text{get}}(i) = O(i), \qquad T_{\text{worst}} = O(n).$$

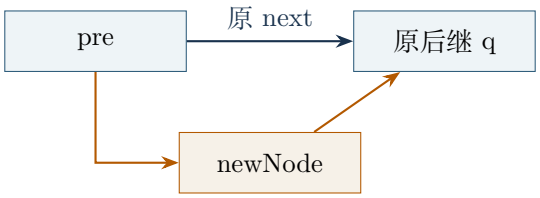
按 ISBN 查找同样是逐结点比较，最坏 $O(n)$。

4. 插入

要在第 i 个位置插入，先找到第 $i-1$ 个结点 `pre`，再执行两句：

`newNode->next = pre->next,  pre->next = newNode.`

顺序不能随意交换。如果先把 `pre->next` 改成新结点，再去读取原来的后继，就会丢失链表后半段地址。



若“前驱结点已经给出”，重连本身是 $O(1)$；但按位置插入还要先找到前驱，整体最坏仍为 $O(n)$。

5. 删除

先令 `q=pre->next`，再

`pre->next=q->next,  delete q.`

`delete q` 不能遗漏，否则每次删除都会造成动态内存泄漏。

6. 两个非递减链表归并

最直观的归并仍是双指针 p_A, p_B。本解答源码采用**复制式归并**：比较两边 ISBN，把较小记录复制为 C 的新结点，因此 A, B 均保持不变。

一题多解：链表归并

解法一（源码采用）：复制结点。新建结果链表，每次复制较小结点的数据。优点是安全、原链表不变；额外空间 $O(m+n)$。

解法二（更体现链表优势）：原地摘链重连。不创建数据结点，直接把 A 或 B 当前较小结点摘下接到结果尾部。因为只是改指针，除头结点外额外空间为 $O(1)$，时间仍为 $O(m+n)$；缺点是会破坏原链表，指针管理也更容易出错。

若实验要求“归并后释放原链表并节省空间”，解法二更合适；若要求保留原数据，解法一更稳妥。

2.1.5 复杂度对比

操作	顺序表	单链表（按位置）	核心原因	推荐理解
初始化	$O(1)$	$O(1)$	仅设置状态	都很快
第 i 个取值	$O(1)$	$O(n)$	数组可地址计算，链表需走指针	顺序表优势
按 ISBN 查找	$O(n)$	$O(n)$	当前均为线性扫描	相同量级
位置插入	$O(n)$	$O(n)$	前者移动元素，后者先找前驱	已知结点时链表更快
位置删除	$O(n)$	$O(n)$	同上	已知结点时链表更快
有序归并	$O(m+n)$	$O(m+n)$	双指针各扫描一次	都是线性时间

2.1.6 易错点

易错点 1：1 基位置与 0 基数组下标混淆

实验接口常把第一个元素记为位置 1，而 C++ 数组第一个下标是 0。因此 `get(i)` 应访问 `data[i-1]`。插入允许的位置范围是 $1 \le i \le n+1$，删除/取值则是 $1 \le i \le n$。

易错点 2：顺序表插入移动方向写反

插入必须从尾部向前移动；删除必须从删除点后方往前覆盖。插入时若从前往后移动，原值会被提前覆盖，后续再也取不回来。

易错点 3：链表插入和删除丢链

任何改 `next` 的操作都应先保存仍需使用的后继地址。删除时还必须 `delete` 被摘除结点；销毁/清空链表也要逐结点释放动态内存。

易错点 4：归并前提没有满足

双指针线性归并的正确性依赖于 A、B 本身已经按同一关键字非递减排列。如果输入表无序，程序虽然能运行，但输出不保证有序。若不确定，应先排序或在建表后验证有序性。

2.1.7 结果检验与测试设计

建议至少覆盖以下边界：

1. 空表初始化后判空；
2. 单元素表的取值、删除；
3. 在表头、表中、表尾插入；
4. 删除表头、表中、表尾；
5. 查找存在 ISBN 与不存在 ISBN；
6. 两个空表归并、一个空表与非空表归并；
7. 两个有序表有交错关键字的归并；
8. 顺序表容量边界。

本次实际编译测试中，令

$$A = (9781, 9783), \qquad B = (9782, 9784),$$

按 ISBN 归并后，顺序表和链表均得到

$$\boxed{9781, 9782, 9783, 9784},$$

符合非递减要求。

第 1 题最终结论

已完成教材信息 `Book` 的顺序表与带头结点单链表实现，覆盖初始化、建表、取值、查找、插入、删除、清空、显示以及两个非递减表按 ISBN 归并。完整可编译 C++17 源码见本节末尾。

2.1.8 实验 1.1 完整 C++ 源码

Listing 2.1: 实验 1.1：顺序表与链表完整实现

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <limits>
using namespace std;

struct Book {
    string isbn;
    string publisher;
    string author;
    double price = 0.0;
};

static Book readBook() {
    Book b;
    cout << "ISBN publisher author price: ";
    cin >> b.isbn >> b.publisher >> b.author >> b.price;
```

```
    return b;
}

static void printBook(const Book& b) {
    cout << left << setw(16) << b.isbn
         << setw(16) << b.publisher
         << setw(16) << b.author
         << fixed << setprecision(2) << b.price << '\n';
}

static void printHeader() {
    cout << left << setw(16) << "ISBN"
         << setw(16) << "Publisher"
         << setw(16) << "Author"
         << "Price\n";
}

class SqList {
private:
    static const int MAXSIZE = 1000;
    Book data[MAXSIZE];
    int length;

public:
    SqList() : length(0) {}

    void clear() { length = 0; }
    int size() const { return length; }
    bool empty() const { return length == 0; }

    bool create(int n) {
        if (n < 0 || n > MAXSIZE) return false;
        length = 0;
        for (int i = 0; i < n; ++i) data[length++] = readBook();
        return true;
    }

    bool get(int i, Book& e) const {
        if (i < 1 || i > length) return false;
        e = data[i - 1];
        return true;
    }

    int locate(const string& isbn) const {
        for (int i = 0; i < length; ++i)
            if (data[i].isbn == isbn) return i + 1;
        return 0;
    }

    bool insert(int i, const Book& e) {
        if (i < 1 || i > length + 1 || length == MAXSIZE) return false;
        for (int j = length; j >= i; --j) data[j] = data[j - 1];
        data[i - 1] = e;
        ++length;
```

```
        return true;
    }

    bool erase(int i, Book& e) {
        if (i < 1 || i > length) return false;
        e = data[i - 1];
        for (int j = i; j < length; ++j) data[j - 1] = data[j];
        --length;
        return true;
    }

    void display() const {
        printHeader();
        for (int i = 0; i < length; ++i) printBook(data[i]);
    }

    // A and B must be nondecreasing by ISBN.
    static bool mergeSorted(const SqList& A, const SqList& B, SqList& C) {
        if (A.length + B.length > MAXSIZE) return false;
        C.clear();
        int i = 0, j = 0;
        while (i < A.length && j < B.length) {
            if (A.data[i].isbn <= B.data[j].isbn) C.data[C.length++] = A.data[i++];
            else C.data[C.length++] = B.data[j++];
        }
        while (i < A.length) C.data[C.length++] = A.data[i++];
        while (j < B.length) C.data[C.length++] = B.data[j++];
        return true;
    }
};

class LinkList {
private:
    struct Node {
        Book data;
        Node* next;
        Node(const Book& b = Book{}, Node* p = nullptr) : data(b), next(p) {}
    };

    Node* head;
    int length;

    Node* nodeBefore(int i) const {
        // i is a 1-based insertion/deletion position; return node at i-1.
        if (i < 1 || i > length + 1) return nullptr;
        Node* p = head;
        for (int k = 1; k < i; ++k) p = p->next;
        return p;
    }

public:
    LinkList() : head(new Node()), length(0) {}
    LinkList(const LinkList&) = delete;
    LinkList& operator=(const LinkList&) = delete;
```

```
126
127     ~LinkList() {
128         clear();
129         delete head;
130     }
131
132     void clear() {
133         Node* p = head->next;
134         while (p) {
135             Node* q = p->next;
136             delete p;
137             p = q;
138         }
139         head->next = nullptr;
140         length = 0;
141     }
142
143     int size() const { return length; }
144     bool empty() const { return length == 0; }
145
146     bool insert(int i, const Book& e) {
147         Node* pre = nodeBefore(i);
148         if (!pre) return false;
149         pre->next = new Node(e, pre->next);
150         ++length;
151         return true;
152     }
153
154     bool create(int n) {
155         if (n < 0) return false;
156         clear();
157         Node* tail = head;
158         for (int i = 0; i < n; ++i) {
159             tail->next = new Node(readBook());
160             tail = tail->next;
161             ++length;
162         }
163         return true;
164     }
165
166     bool get(int i, Book& e) const {
167         if (i < 1 || i > length) return false;
168         Node* p = head->next;
169         for (int k = 1; k < i; ++k) p = p->next;
170         e = p->data;
171         return true;
172     }
173
174     int locate(const string& isbn) const {
175         Node* p = head->next;
176         int i = 1;
177         while (p) {
178             if (p->data.isbn == isbn) return i;
179             p = p->next;
```

```
            ++i;
        }
        return 0;
    }

    bool erase(int i, Book& e) {
        if (i < 1 || i > length) return false;
        Node* pre = nodeBefore(i);
        Node* q = pre->next;
        e = q->data;
        pre->next = q->next;
        delete q;
        --length;
        return true;
    }

    void display() const {
        printHeader();
        for (Node* p = head->next; p; p = p->next) printBook(p->data);
    }

    // Copy-merge: A and B are unchanged; C receives a sorted result.
    static void mergeSorted(const LinkList& A, const LinkList& B, LinkList& C) {
        C.clear();
        Node* pa = A.head->next;
        Node* pb = B.head->next;
        Node* tail = C.head;

        while (pa && pb) {
            const Book* chosen;
            if (pa->data.isbn <= pb->data.isbn) {
                chosen = &pa->data;
                pa = pa->next;
            } else {
                chosen = &pb->data;
                pb = pb->next;
            }
            tail->next = new Node(*chosen);
            tail = tail->next;
            ++C.length;
        }
        while (pa) {
            tail->next = new Node(pa->data);
            tail = tail->next;
            pa = pa->next;
            ++C.length;
        }
        while (pb) {
            tail->next = new Node(pb->data);
            tail = tail->next;
            pb = pb->next;
            ++C.length;
        }
    }
```

```
};

static void runSqList() {
    SqList L;
    for (;;) {
        cout << "\n[Sequential List] 1.Create 2.Get 3.Find 4.Insert 5.Delete "
             << "6.Merge 7.Display 8.Clear 0.Exit\nChoice: ";
        int op; cin >> op;
        if (op == 0) break;
        if (op == 1) {
            int n; cout << "n: "; cin >> n;
            cout << (L.create(n) ? "Created.\n" : "Create failed.\n");
        } else if (op == 2) {
            int i; Book e; cout << "position: "; cin >> i;
            if (L.get(i, e)) { printHeader(); printBook(e); }
            else cout << "Invalid position.\n";
        } else if (op == 3) {
            string x; cout << "ISBN: "; cin >> x;
            int p = L.locate(x);
            cout << (p ? "Position = " + to_string(p) : "Not found") << '\n';
        } else if (op == 4) {
            int i; cout << "position: "; cin >> i;
            Book e = readBook();
            cout << (L.insert(i, e) ? "Inserted.\n" : "Insert failed.\n");
        } else if (op == 5) {
            int i; Book e; cout << "position: "; cin >> i;
            if (L.erase(i, e)) { cout << "Deleted:\n"; printHeader(); printBook(e); }
            else cout << "Delete failed.\n";
        } else if (op == 6) {
            SqList B, C;
            int n; cout << "Input sorted B, n: "; cin >> n; B.create(n);
            if (SqList::mergeSorted(L, B, C)) { cout << "Merged result:\n"; C.display();
    }
            else cout << "Merge failed.\n";
        } else if (op == 7) L.display();
        else if (op == 8) { L.clear(); cout << "Cleared.\n"; }
    }
}

static void runLinkList() {
    LinkList L;
    for (;;) {
        cout << "\n[Linked List] 1.Create 2.Get 3.Find 4.Insert 5.Delete "
             << "6.Merge 7.Display 8.Clear 0.Exit\nChoice: ";
        int op; cin >> op;
        if (op == 0) break;
        if (op == 1) {
            int n; cout << "n: "; cin >> n;
            cout << (L.create(n) ? "Created.\n" : "Create failed.\n");
        } else if (op == 2) {
            int i; Book e; cout << "position: "; cin >> i;
            if (L.get(i, e)) { printHeader(); printBook(e); }
            else cout << "Invalid position.\n";
        } else if (op == 3) {
```

```
            string x; cout << "ISBN: "; cin >> x;
            int p = L.locate(x);
            cout << (p ? "Position = " + to_string(p) : "Not found") << '\n';
        } else if (op == 4) {
            int i; cout << "position: "; cin >> i;
            Book e = readBook();
            cout << (L.insert(i, e) ? "Inserted.\n" : "Insert failed.\n");
        } else if (op == 5) {
            int i; Book e; cout << "position: "; cin >> i;
            if (L.erase(i, e)) { cout << "Deleted:\n"; printHeader(); printBook(e); }
            else cout << "Delete failed.\n";
        } else if (op == 6) {
            LinkList B, C;
            int n; cout << "Input sorted B, n: "; cin >> n; B.create(n);
            LinkList::mergeSorted(L, B, C);
            cout << "Merged result:\n"; C.display();
        } else if (op == 7) L.display();
        else if (op == 8) { L.clear(); cout << "Cleared.\n"; }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cout << "Choose implementation: 1.Sequential list 2.Linked list: ";
    int mode; cin >> mode;
    if (mode == 1) runSqList();
    else if (mode == 2) runLinkList();
    return 0;
}
```

2.2 实验 1.2：栈的操作

2.2.1 考查知识点与总体设计

栈是只允许在一端进行插入和删除的线性结构，这一端称为**栈顶**。它满足

$$\boxed{\text{后进先出（LIFO, Last In First Out）}}.$$

Stanford CS106B 的栈课程同样把栈描述为只从顶部加入和移除元素的 LIFO 结构；C++ 标准库 `std::stack` 也明确提供这一语义。[8, 10]

本实验要求分别实现顺序栈和链栈，然后把“后进先出”特性用于括号匹配、进制转换、表达式转换和表达式求值。

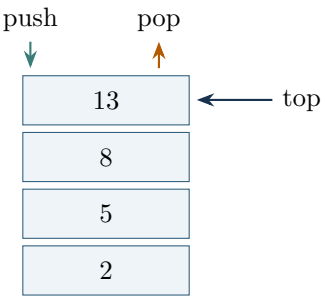
2.2.2 顺序栈与链栈结构

顺序栈

用数组 `data[MAXSIZE]` 保存栈元素，并用 `topIndex` 指向当前栈顶下标：

$$\text{空栈：}\quad \texttt{topIndex} = -1.$$

入栈：先将 `topIndex` 加 1，再写入；出栈：先读栈顶，再将 `topIndex` 减 1。



只要未满，入栈、出栈、取栈顶、判空都是 $O(1)$。

链栈

链栈把**链表表头直接作为栈顶**。`topPtr` 指向当前栈顶结点：

$$\text{入栈：}\quad p \to next = topPtr,\ topPtr = p;$$

$$\text{出栈：}\quad p = topPtr,\ topPtr = topPtr \to next,\ delete\ p.$$

无需遍历，因此入栈、出栈也都是 $O(1)$，且没有固定数组容量上限（只受可用内存限制）。

2.2.3 基本操作逐项说明

初始化

顺序栈设 `topIndex=-1`；链栈设 `topPtr=nullptr`、元素计数为 0。

入栈 Push

顺序栈必须先检查“满栈”；链栈动态申请新结点。成功后新元素成为唯一可直接访问的栈顶元素。

出栈 Pop

必须先判空。若空栈仍执行读取，顺序栈会访问非法下标，链栈会解引用空指针。

取栈顶 Top

读取但不删除元素。因此执行 `top` 后栈的长度不变，执行 `pop` 后长度减 1。

判空 Empty

顺序栈条件为 `topIndex == -1`；链栈条件为 `topPtr == nullptr`。

2.2.4 扩展 1：十进制整数转八进制

所给操作演示包含“10 进制数字转换为 8 进制”。它是栈的经典应用，所以一并给出。

对十进制非负整数 N 连续做除 8 取余：

$$N = 8q + r, \qquad 0 \le r < 8.$$

最先得到的是**最低位**，最后得到的是**最高位**。输出顺序恰好与产生顺序相反，因此把余数依次压栈，再逐个出栈即可。

以 12_{10} 为例：

$$12 \div 8 = 1 \text{ 余} 4, \qquad 1 \div 8 = 0 \text{ 余} 1.$$

余数产生顺序为 $4, 1$，入栈后出栈顺序为 $1, 4$，所以

$$\boxed{12_{10} = 14_8}.$$

若 N 有 k 位八进制数字，循环次数为 $k = \lfloor \log_8 N \rfloor + 1$，故时间和栈空间均为 $O(\log N)$。

2.2.5 括号匹配：为什么一定适合用栈？

合法括号具有“最近打开的括号必须最先闭合”的嵌套规律。例如：

```
([{}])
```

读取到最内层 `{` 后，必须先遇到 `}`；这正是 LIFO。

算法如下：

1. 从左到右扫描字符串；
2. 遇到左括号 `(`、`[`、`{`，直接入栈；
3. 遇到右括号时，若栈空，则缺少左括号，立即失败；
4. 否则弹出栈顶，检查是否为对应类型；类型不一致立即失败；
5. 扫描结束后，若栈仍非空，说明有左括号未闭合；只有最终空栈才匹配成功。

扫描字符	操作	栈（底 → 顶）	判断
(	push	(	暂时合法
[	push	([	暂时合法
{	push	([{	暂时合法
}	pop {	([	类型匹配
]	pop [	(	类型匹配
)	pop (	空	类型匹配

每个括号最多入栈一次、出栈一次，因此长度为 n 的字符串：

$$\boxed{T(n) = O(n)}, \qquad \boxed{S(n) = O(n)}.$$

易错点：只统计左右括号数量是不够的

字符串 ([)] 中左右括号数量完全相等，但嵌套顺序错误。正确算法必须检查“当前右括号是否与最近尚未闭合的左括号同类型”，这正是栈顶匹配。

2.2.6 中缀表达式与后缀表达式

中缀表达式把运算符写在两个操作数中间，例如

$$2 + 3 \times 4.$$

计算机必须额外处理优先级和括号。后缀表达式（逆波兰式）把运算符写在操作数之后：

$$2 \quad 3 \quad 4 \quad * \quad +.$$

此时运算顺序已编码在符号次序里，不再需要括号和优先级判断。Stanford 的课程材料说明了后缀表达式可从左到右用一个栈直接求值。[8]

2.2.7 中缀转后缀：运算符栈算法

本解答采用 Dijkstra 的 shunting-yard 思想。Emory 的教学材料明确给出：操作数直接输出；运算符进入栈，并根据优先级弹出栈顶运算符；括号控制弹栈范围。[9]

对本实验四则运算：

$$\mathrm{prec}(*) = \mathrm{prec}(/) = 2, \qquad \mathrm{prec}(+) = \mathrm{prec}(-) = 1.$$

且四个运算符均按左结合处理。

规则

1. 读到数字：直接加入后缀输出；
2. 读到左括号 (：入运算符栈；
3. 读到右括号)：持续弹栈并输出，直到遇到左括号；左右括号本身都不输出；
4. 读到运算符 op：只要栈顶也是运算符且其优先级不低于 op，就先弹出栈顶到输出；再把 op 入栈；
5. 输入结束：把剩余运算符依次弹出输出；若此时仍遇到左括号，说明括号不匹配。

完整例子： $2+3\times4$

当前 token	运算符栈（底 → 顶）	后缀输出
2	空	2
+	+	2
3	+	2 3
*	+ *	2 3
4	+ *	2 3 4
结束	空	2 3 4 * +

所以

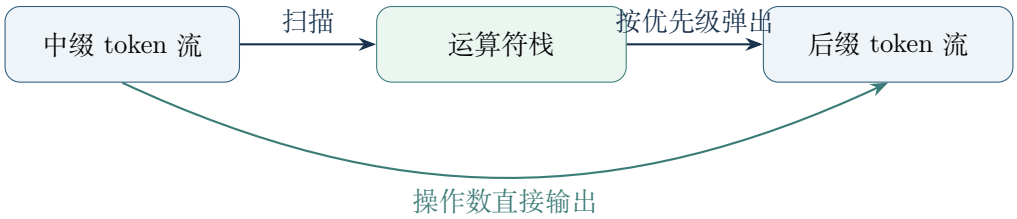
$$\boxed{2+3\times4 \longrightarrow 2\ 3\ 4\ *\ +}.$$

带括号例子： $(2+3)\times4-12/3$

逐步处理后得到

$$\boxed{2\ 3\ +\ 4\ *\ 12\ 3\ /\ -}.$$

括号只控制出栈次序，不会进入后缀结果。



每个 token 最多入栈一次、出栈一次，故转换时间为 $O(n)$，运算符栈最坏空间为 $O(n)$。

2.2.8 后缀表达式求值

算法只需要一个**数值栈**：

1. 从左到右扫描后缀 token；
2. 数字直接入栈；
3. 运算符出现时，连续弹出两个数；
4. **第一次弹出的是右操作数** rhs，第二次弹出的是左操作数 lhs；
5. 计算 $lhs\ op\ rhs$，把结果重新压栈；
6. 扫描结束时必须恰好剩一个数，它就是最终答案。

Stanford 对后缀求值特别强调了减法和除法中弹栈次序：先弹的是右操作数，后弹的是左操作数。[8]

以

$$2\ 3\ 4\ *\ +$$

为例：

1. 2 入栈：$[2]$；
2. 3 入栈：$[2,3]$；
3. 4 入栈：$[2,3,4]$；
4. 读到 $*$：弹出 $rhs=4$、$lhs=3$，计算 $3\times4=12$，入栈得到 $[2,12]$；

5. 读到 +：弹出 $rhs = 12$、$lhs = 2$，计算 $2 + 12 = 14$，入栈得到 [14]；
6. 扫描结束，唯一栈顶即 14。

因此

$$\boxed{2 + 3 \times 4 = 14}.$$

易错点：减法与除法操作数顺序

后缀 `12 3 /`：第一次弹出 3，第二次弹出 12，必须计算 12/3，而不是 3/12。加法与乘法交换后结果不变，所以这个错误常被它们“掩盖”。

2.2.9 异常输入与健壮性处理

一个能通过基本演示的计算器不等于一个可靠计算器。源码额外检查：

1. 非法字符；
2. 连续两个数字但中间没有运算符；
3. 二元运算符缺少左/右操作数；
4. 左右括号不匹配；
5. 后缀求值时操作数不足；
6. 最终栈内不是恰好一个结果；
7. 除数为 0。

本实验规定只支持二元 `+ - * /`。因此 `-3+2` 这类**一元负号**不在必做范围内；若要扩展，应在词法/语法阶段专门区分一元负号，而不是把它直接当作普通减法处理。

2.2.10 一题多解：表达式计算的两种栈方案

解法一：中缀 → 后缀 → 求值（本解答采用）

先用运算符栈完成转换，再用数值栈计算后缀。优点是两个阶段职责清晰、便于调试，而且中间后缀表达式本身可以输出作为实验结果；最适合课程教学。

解法二：双栈直接计算中缀

维护“运算符栈”和“操作数栈”，扫描中缀表达式时依据优先级决定是否立即进行一次运算。这样不必保存完整后缀式，空间常数更小一些；但语法判断、括号处理和运算时机都混在同一流程中，对初学者更难调试。

方法比较：

- 教学清晰度：解法一更好；
- 需要展示“后缀转换结果”：只能选解法一或至少显式生成后缀；
- 单次直接求值：双栈法可少保存一份后缀序列；
- 本实验要求明确包含“中缀转后缀”和“后缀计算”，因此采用解法一最契合题意。

2.2.11 复杂度汇总

功能	时间复杂度	额外空间	说明
顺序栈 push/pop/top	$O(1)$	$O(1)$	固定数组内操作
链栈 push/pop/top	$O(1)$	每次 push 新结点	表头即栈顶
十进制转八进制	$O(\log N)$	$O(\log N)$	每次除 8
多括号匹配	$O(n)$	$O(n)$	每字符至多入/出栈一次
中缀转后缀	$O(n)$	$O(n)$	每 token 至多入/出运算符栈一次
后缀求值	$O(n)$	$O(n)$	每 token 处理一次

2.2.12 结果检验

完整源码使用

```
g++ -std=c++17 -Wall -Wextra -pedantic
```

编译通过，并实际验证：

- $12_{10} \to 14_8$；
- `([{}])` 判断为匹配；
- `([)]` 判断为不匹配；
- `2+3*4` 转换为 `2 3 4 * +`，结果为 14；
- `(2+3)*4-12/3` 转换为 `2 3 + 4 * 12 3 / -`，结果为 16；
- `10/(5-5)` 在求值阶段正确识别除零并拒绝给出数值结果。

第 2 题最终结论

已实现顺序栈与链栈的初始化、入栈、出栈、取栈顶、判空、清空，并完成十进制转八进制、多类型括号匹配、中缀转后缀和后缀四则运算求值。算法均遵循栈的 LIFO 规律，且针对括号错误、操作数不足、非法输入和除零做了必要检查。

2.2.13 实验 1.2 完整 C++ 源码

Listing 2.2: 实验 1.2：顺序栈、链栈与表达式计算器完整实现

```
#include <cmath>
#include <cctype>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

template <class T>
class SqStack {
private:
```

```
    static const int MAXSIZE = 1000;
    T data[MAXSIZE];
    int topIndex;

public:
    SqStack() : topIndex(-1) {}
    void clear() { topIndex = -1; }
    bool empty() const { return topIndex == -1; }
    bool full() const { return topIndex == MAXSIZE - 1; }
    int size() const { return topIndex + 1; }

    bool push(const T& x) {
        if (full()) return false;
        data[++topIndex] = x;
        return true;
    }

    bool pop(T& x) {
        if (empty()) return false;
        x = data[topIndex--];
        return true;
    }

    bool top(T& x) const {
        if (empty()) return false;
        x = data[topIndex];
        return true;
    }

    vector<T> valuesBottomToTop() const {
        vector<T> result;
        for (int i = 0; i <= topIndex; ++i) result.push_back(data[i]);
        return result;
    }
};

template <class T>
class LinkStack {
private:
    struct Node {
        T data;
        Node* next;
        Node(const T& x, Node* p) : data(x), next(p) {}
    };

    Node* topPtr;
    int count;

public:
    LinkStack() : topPtr(nullptr), count(0) {}
    LinkStack(const LinkStack&) = delete;
    LinkStack& operator=(const LinkStack&) = delete;

    ~LinkStack() { clear(); }
```

```
68
69      void clear() {
70          while (topPtr) {
71              Node* p = topPtr;
72              topPtr = topPtr->next;
73              delete p;
74          }
75          count = 0;
76      }
77
78      bool empty() const { return topPtr == nullptr; }
79      int size() const { return count; }
80
81      bool push(const T& x) {
82          topPtr = new Node(x, topPtr);
83          ++count;
84          return true;
85      }
86
87      bool pop(T& x) {
88          if (empty()) return false;
89          Node* p = topPtr;
90          x = p->data;
91          topPtr = p->next;
92          delete p;
93          --count;
94          return true;
95      }
96
97      bool top(T& x) const {
98          if (empty()) return false;
99          x = topPtr->data;
100         return true;
101     }
102
103     vector<T> valuesBottomToTop() const {
104         vector<T> reversed;
105         for (Node* p = topPtr; p; p = p->next) reversed.push_back(p->data);
106         return vector<T>(reversed.rbegin(), reversed.rend());
107     }
108 };
109
110 static bool isOpen(char c) { return c == '(' || c == '[' || c == '{'; }
111 static bool isClose(char c) { return c == ')' || c == ']' || c == '}'; }
112 static bool pairMatches(char a, char b) {
113     return (a == '(' && b == ')') || (a == '[' && b == ']') || (a == '{' && b == '}');
114 }
115
116 template <template <class> class StackT>
117 bool bracketsMatched(const string& s) {
118     StackT<char> st;
119     for (char c : s) {
120         if (isOpen(c)) {
121             if (!st.push(c)) return false;
```

```
        } else if (isClose(c)) {
            char t;
            if (!st.pop(t) || !pairMatches(t, c)) return false;
        }
    }
    return st.empty();
}

template <template <class> class StackT>
string decimalToOctal(unsigned int n) {
    if (n == 0) return "0";
    StackT<int> st;
    while (n > 0) {
        st.push(static_cast<int>(n % 8));
        n /= 8;
    }
    string result;
    int d;
    while (st.pop(d)) result.push_back(static_cast<char>('0' + d));
    return result;
}

static bool isOperatorToken(const string& tok) {
    return tok.size() == 1 && (tok[0] == '+' || tok[0] == '-' || tok[0] == '*' || tok[0]
    == '/');
}

static bool isOperatorChar(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

static int precedence(char c) {
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

static bool tokenize(const string& infix, vector<string>& tokens) {
    tokens.clear();
    for (size_t i = 0; i < infix.size();) {
        unsigned char uc = static_cast<unsigned char>(infix[i]);
        if (isspace(uc)) { ++i; continue; }
        if (isdigit(uc) || infix[i] == '.') {
            size_t j = i;
            int dots = 0;
            while (j < infix.size()) {
                unsigned char uj = static_cast<unsigned char>(infix[j]);
                if (isdigit(uj)) { ++j; continue; }
                if (infix[j] == '.') { ++dots; ++j; continue; }
                break;
            }
            string num = infix.substr(i, j - i);
            if (dots > 1 || num == ".") return false;
            tokens.push_back(num);
```

```
            i = j;
        } else if (isOperatorChar(infix[i]) || infix[i] == '(' || infix[i] == ')') {
            tokens.push_back(string(1, infix[i]));
            ++i;
        } else {
            return false;
        }
    }
    return !tokens.empty();
}

template <template <class> class StackT>
bool infixToPostfix(const string& infix, vector<string>& postfix) {
    vector<string> tokens;
    if (!tokenize(infix, tokens)) return false;

    postfix.clear();
    StackT<char> ops;
    bool expectOperand = true;

    for (const string& tok : tokens) {
        if (!isOperatorToken(tok) && tok != "(" && tok != ")") {
            if (!expectOperand) return false; // no implicit multiplication in this
    experiment
            postfix.push_back(tok);
            expectOperand = false;
        } else if (tok == "(") {
            if (!expectOperand) return false;
            ops.push('(');
            expectOperand = true;
        } else if (tok == ")") {
            if (expectOperand) return false;
            char t;
            bool foundLeft = false;
            while (ops.top(t)) {
                ops.pop(t);
                if (t == '(') { foundLeft = true; break; }
                postfix.push_back(string(1, t));
            }
            if (!foundLeft) return false;
            expectOperand = false;
        } else {
            if (expectOperand) return false; // unary +/- is outside the required scope
            char cur = tok[0], t;
            while (ops.top(t) && t != '(' && precedence(t) >= precedence(cur)) {
                ops.pop(t);
                postfix.push_back(string(1, t));
            }
            ops.push(cur);
            expectOperand = true;
        }
    }

    if (expectOperand) return false;
```

```
    char t;
    while (ops.pop(t)) {
        if (t == '(') return false;
        postfix.push_back(string(1, t));
    }
    return true;
}

static bool parseNumber(const string& s, double& x) {
    try {
        size_t pos = 0;
        x = stod(s, &pos);
        return pos == s.size();
    } catch (...) {
        return false;
    }
}

template <template <class> class StackT>
bool evaluatePostfix(const vector<string>& postfix, double& answer) {
    StackT<double> st;
    for (const string& tok : postfix) {
        if (!isOperatorToken(tok)) {
            double x;
            if (!parseNumber(tok, x) || !st.push(x)) return false;
        } else {
            double rhs, lhs;
            if (!st.pop(rhs) || !st.pop(lhs)) return false;
            double r = 0.0;
            switch (tok[0]) {
                case '+': r = lhs + rhs; break;
                case '-': r = lhs - rhs; break;
                case '*': r = lhs * rhs; break;
                case '/':
                    if (fabs(rhs) < 1e-12) return false;
                    r = lhs / rhs; break;
            }
            if (!st.push(r)) return false;
        }
    }
    if (st.size() != 1) return false;
    return st.pop(answer);
}

static string joinTokens(const vector<string>& v) {
    ostringstream out;
    for (size_t i = 0; i < v.size(); ++i) {
        if (i) out << ' ';
        out << v[i];
    }
    return out.str();
}

template <template <class> class StackT>
```

```
282 void basicStackDemo() {
283     StackT<int> st;
284     cout << "Input integers; enter -999999 to stop: ";
285     int x;
286     while (cin >> x && x != -999999) st.push(x);
287     cout << "size = " << st.size() << ", empty = " << boolalpha << st.empty() << '\n';
288     auto vals = st.valuesBottomToTop();
289     cout << "bottom -> top: ";
290     for (int v : vals) cout << v << ' ';
291     cout << '\n';
292     if (st.top(x)) cout << "top = " << x << '\n';
293     if (st.pop(x)) cout << "pop = " << x << '\n';
294     cout << "size after pop = " << st.size() << '\n';
295     st.clear();
296     cout << "cleared, empty = " << st.empty() << '\n';
297 }
298 
299 int main() {
300     ios::sync_with_stdio(false);
301     cin.tie(nullptr);
302 
303     for (;;) {
304         cout << "\n1.Basic sequential stack\n"
305              << "2.Basic linked stack\n"
306              << "3.Decimal -> octal\n"
307              << "4.Bracket matching: (), [], {}\n"
308              << "5.Infix -> postfix -> evaluate (+ - * /)\n"
309              << "0.Exit\nChoice: ";
310         int op;
311         if (!(cin >> op) || op == 0) break;
312         if (op == 1) {
313             basicStackDemo<SqStack>();
314         } else if (op == 2) {
315             basicStackDemo<LinkStack>();
316         } else if (op == 3) {
317             unsigned int n; cout << "decimal integer: "; cin >> n;
318             cout << "octal = " << decimalToOctal<SqStack>(n) << '\n';
319         } else if (op == 4) {
320             cout << "string: ";
321             string s; cin >> ws; getline(cin, s);
322             cout << (bracketsMatched<SqStack>(s) ? "matched\n" : "not matched\n");
323         } else if (op == 5) {
324             cout << "infix: ";
325             string expr; cin >> ws; getline(cin, expr);
326             vector<string> postfix;
327             double ans;
328             if (!infixToPostfix<SqStack>(expr, postfix)) {
329                 cout << "invalid infix expression\n";
330             } else {
331                 cout << "postfix = " << joinTokens(postfix) << '\n';
332                 if (evaluatePostfix<SqStack>(postfix, ans))
333                     cout << "answer = " << setprecision(12) << ans << '\n';
334                 else
335                     cout << "postfix evaluation failed\n";
```

```
336                 }
337             }
338         }
339         return 0;
340 }
```

第 3 章

参考资料与核验说明

核验结论

本解答先根据上传实验指导截图重建正式题目，再独立完成数据结构设计和 C++ 实现。网络资料只用于核验以下知识点：链表的结点/指针模型、栈的 LIFO 语义、中缀转后缀的栈算法、后缀表达式求值规则以及标准容器的存储/复杂度描述；没有直接复制网络作业答案。

参考文献

[1] 游洪跃等. 《数据结构与算法》C++ 版. 清华大学出版社.（上传材料“适用教材”）

[2] 游洪跃等.《数据结构与算法实验和课程设计》（第 2 版）. 清华大学出版社.（上传材料“参考书”）

[3] Sartaj Sahni 著，王立柱、刘志红译. 《数据结构算法与应用：C++ 语言描述》.（上传材料“参考书”）

[4] 殷人昆. 《数据结构（用面向对象方法与 C++ 描述）》（第 2 版）. 清华大学出版社.（上传材料“参考书”）

[5] 张铭，王腾蛟，赵海燕. 《数据结构与算法》. 高等教育出版社.（上传材料“参考书”）

[6] MIT OpenCourseWare, 6.006 Introduction to Algorithms, Linked List Sequence / course materials. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms/. Accessed 2026-09-15.

[7] Stanford University CS106B, Linked Lists lecture materials. https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1206/lectures/linked-lists1/. Accessed 2026-09-15.

[8] Stanford University CS106B, Stacks and Queues; supplementary postfix notation. https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1266/lectures/05-stack-queue/. Accessed 2026-09-15.

[9] Emory University Math/CS Center, The Shunting Yard Algorithm. https://mathcenter.oxford.emory.edu/site/cs171/shuntingYardAlgorithm/. Accessed 2026-09-15.

[10] cppreference, `std::stack`. https://en.cppreference.com/cpp/container/stack. Accessed 2026-09-15.

[11] cppreference, `std::vector`. https://en.cppreference.com/cpp/container/vector. Accessed 2026-09-15.

[12] cppreference, `std::list`. https://en.cppreference.com/cpp/container/list. Accessed 2026-09-15.

源码验证环境

- 语言标准：C++17；
- 编译参数：`-Wall -Wextra -pedantic`；
- 两份源码均已实际编译并运行样例；
- PDF 使用 XeLaTeX 编译，并在生成后逐页渲染检查版面。